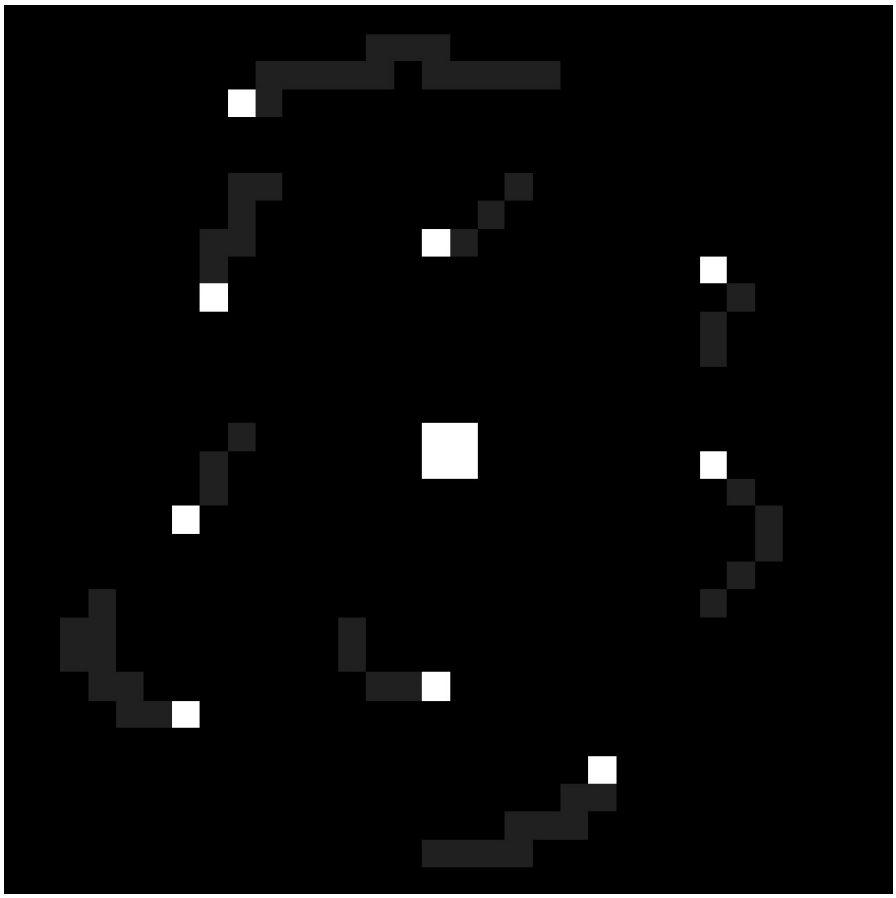
On implementing an N-Body simulation



ShadowNetter

Physics

x → X position in 2D plane
y → Y position in 2D plane

vx → X velocity
vy → Y velocity

ax → X acceleration
ay → Y acceleration

g → Gravitational constant

Δt → Delta Time: time between calculations

force → Pull between object

Physics

Object Movement:

Every cycle:

$x = x + vx * \Delta t$
$y = y + vy * \Delta t$

$vx = vx + ax * \Delta t$
$vy = vy + ay * \Delta t$

To calculate acceleration, one must calculate acceleration between every object and increment to total acceleration.

To calculate acceleration for Object i and current Object s:

$g = 1.0$

$ax = force\_x / s.mass$
$ay = force\_y / s.mass$

$force\_x = force * dx / distance$
$force\_y = force * dy / distance$

$force = g * (i.mass * s.mass / distance^2)$

$distance = \sqrt{(dx^2 + dy^2)}$

$dx = i.x - s.x$
$dy = i.y - s.y$

then add ax and ay to total ax and total ay

Physics

Object Collision (Inelastic Collision):

To calculate collision, one must define a collision radius defined by mass and an arbitrary scalar.

Every cycle:

For Object i and current Object s:

radius = radius_scalar * s.mass

dx = i.x - s.x
dy = i.y - s.y

distance = √ (dx^2 + dy^2)

if distance is less or equal to radius

create new Object n

n.mass = i.mass + s.mass

n.vx = (s.mass * s.vx + i.mass * i.vx) / (s.mass + i.mass)
n.vy = (s.mass * s.vy + i.mass * i.vy) / (s.mass + i.mass)

n.x = (s.mass * s.x + i.mass * i.x) / (s.mass + i.mass)
n.y = (s.mass * s.y + i.mass * i.y) / (s.mass + i.mass)

Replace Object s's data for Object n's data
and invalidate Object i

s.x = n.x
s.y = n.y
s.vx = n.vx
s.vy = n.vy

i.valid = false

Code

To implement an example of this simulation, the following pseudocode may help.

```
        // create Object structure

        struct Object {
            double x;
            double y;
            double ax;
            double ay;
            double vx;
            double vy;
            double mass;
            bool valid;
        }

    // define arbitrary variables

    const int COUNT = 10;
    const int FPS = 60;
    const int delay = 1000 / FPS;
    const double G = 1.0;
    const double DT = 0.1;

    fn main() {

        // initialize space
        Object space[COUNT];
        initialize_space(space);

        bool running = true;
        while (running) {
            clear_screen();

            calculate(space);
            collisions(space);

            draw();

            wait(delay);
        }
    }
```

Code

```
fn calculate(Object space[COUNT]) {
    for every object i in space {

        // check if object is valid
        if i.valid == false { continue; }

        double total_ax = 0;
        double total_ay = 0;

        for every object j in space {
            // check if object is valid and that object is not self
            if j.valid == false or j == i { continue; }

            double dx = j.x - i.x;
            double dy = j.y - i.y;
            double distance = √ (dx² + dy²);

            // avoid divistion by zero
            if distance < 0.1 { distance = 0.1 }

            double force = G * j.mass * i.mass / distance²;
            double force_x = force * dx / distance;
            double force_y = force * dy / distance;

            double ax = force_x / i.mass;
            double ay = force_y / i.mass;

            total_ax = total_ax + ax;
            total_ay = total_ay + ay;
        }

        // update position
        i.vx = i.vx + total_ax * DT;
        i.vy = i.vy + total_ay * DT;

        i.x = i.x + i.vx * DT;
        i.y = i.y + i.vy * DT;
    }
}
```

Code

```
fn collisions(Object space[COUNT]) {
    double radius_scalar = 0.0003;

    for every Object i {

        if i.valid == false { continue; }

        for every Object j {

            if j.valid == false { continue; }

            double radius = radius_scalar * i.mass;

            double dx = j.x - i.x;
            double dy = j.y - i.y;
            double distance = √ (dx² + dy²);

            // avoid divistion by zero
            if distance < 0.1 { distance = 0.1 }

            if distance ⩽ radius {
                double n_mass = i.mass + j.mass;

                double n_vx = i.mass * i.vx + j.mass * j.vx / ( i.mass + j.mass );
                double n_vy = i.mass * i.vy + j.mass * j.vy / ( i.mass + j.mass );

                double n_x = i.mass * i.x + j.mass * j.x / ( i.mass + j.mass );
                double n_y = i.mass * i.y + j.mass * j.y / ( i.mass + j.mass );

                i.x = n_x;
                i.y = n_y;
                i.vx = n_vx;
                i.vy = n_vy;

                j.valid = false;
                break;
            }
        }
    }
}
```